SOFTWARE REQUIREMENTS SPECIFICATION

RealmForge

Team 20
Aeolus Software

Andy Huynh
Owen Johnson
Christopher Ludwig
Andreas Sideris
Humna Saeed

Revision 0: October 20, 2026

Revision History

Name	Date	Reason For Changes	Version
Andy Huynh	2026-10-05	First Draft	1.0
Andy Huynh	2026-10-13	Second draft of G4	1.0
Andreas Sideris	2026-10-10	P3 and P4 drafts	1.0
Andreas Sideris	2026-10-12	E3 draft	1.0
Andreas Sideris	2026-10-13	E2 draft	1.0
Owen Johnson	2026-10-14	G7, S1 and P2 draft	1.0
Andreas Sideris	2026-10-15	AI disclosure and Reflection updated	1.0
Owen Johnson	2026-10-15	Reflection updated	1.0
Andreas Sideris	2026-10-16	S4 draft	1.0
Humna Saeed	2026-10-16	E4, E5, and E6 Draft	1.0
Owen Johnson	2026-10-19	P6 Updated	1.0
Owen Johnson	2026-10-17	G6 Draft	1.0

Document Template and Conventions

Document Template:

This document follows the requirements documentation structure presented in the Handbook of requirements and business analysis, by Bertrand Meyer. The document template has been modified with the following changes to better suit the project's needs:

- A requirement and behavior artifact traceability table has been added to section P.7, to enable further traceability between essential requirements and behavior.

Requirement and Behavior Artifact Conventions:

All formal requirements in this document will use the following conventions:

- Functional Requirements are given the unique identifier "SFR-XXX", meaning "Software Functional Requirement", with "XXX" being a unique numerical identifier.
- Non-Functional Requirements are given the unique identifier "SNFR-XXX", meaning "Software Non-Functional Requirement", with "XXX" being a unique numerical identifier.
- Environmental Requirements are given the unique identifier "ENV-XXX", meaning "Environmental Requirement", with "XXX" being a unique numerical identifier.
- Mandatory Requirements that must be satisfied at the conclusion of the project (except those that are denoted as "stretch" requirements to be satisfied in a future revision of the software) use the keyword "**shall**" to express a requirement. For example: "The system shall ___". The majority of requirements presented in this document will follow this convention.
- Non-Mandatory Requirements that express a recommendation for the implementation of the system use the keyword "**should**" to express these recommendations. For example: "The system should ___".

Behavior Artifacts in this document will use the following conventions:

- High-level Use Cases are given the unique identifier "UC-XXX", meaning "Use Case", with "XXX" being a unique numerical identifier.

- Detailed User Scenarios are given the unique identifier "US-XXX", meaning "User Scenario", with "XXX" being a unique numerical identifier.

The following traceability relationships will be used between requirements and behavior artifacts:

- A requirement can "**satisfy**" another requirement or be "**satisfied by**" another requirement. This relationship describes when a requirement provides further detail and more criteria to **satisfy** another, such as a functional requirement providing more detail to satisfy a non-functional requirement.
- A requirement can "**refine**" a use case or user scenario, and a use case or user scenario can be "**refined by**" a requirement. This relationship describes when a requirement expresses a mandatory need to implement the behavior described in a use case or user scenario in a specific way, by **refining** the behavior.

Contents

Goals book **6**
G.1 Context and overall objective . . . 6
G.2 Current situation . . . 6
G.3 Expected benefits . . . 7
G.4 Functionality overview . . . 7
G.5 High-level usage scenarios . . . 8
G.6 Limitations and exclusions . . . 11
G.7 Stakeholders and requirements sources . . . 11

Environment book **12**
E.1 Glossary . . . 12
E.2 Components . . . 15
E.3 Constraints . . . 16
E.4 Assumptions . . . 17
E.5 Effects . . . 17
E.6 Invariants . . . 18

System book **20**
S.1 Components . . . 20
S.2 Functionality . . . 21
S.3 Interfaces . . . 31
S.4 Detailed usage scenarios . . . 31
S.5 Prioritization . . . 39
S.6 Verification and acceptance criteria . . . 39

Project book **41**
P.1 Roles and personnel . . . 41
P.2 Imposed technical choices . . . 43
P.3 Schedule and milestones . . . 43
P.4 Tasks and deliverables . . . 44
P.5 Required technology elements . . . 46
P.6 Risks and mitigation analysis . . . 47
P.7 Requirements process and report . . . 47

Appendix **48**
Appendix — Generative AI Use Disclosure . . . 48
Appendix — Reflection . . . 49

Goals

Contents

G.1 Context and overall objective 6
G.2 Current situation 6
G.3 Expected benefits 7
G.4 Functionality overview 7
G.5 High-level usage scenarios 8
G.6 Limitations and exclusions 11
G.7 Stakeholders and requirements sources 11

G.1 Context and overall objective

Context and Overall Objectives

In recent years, the popularity of games in the roguelite genre has massively increased as seen in large titles such as *Hades*, *Slay the Spire*, *Balatro*, and more. The surge in popularity of these types of games has caused the development of many new projects aiming to follow in the footsteps of these major successes; however, in an attempt to quickly recreate their success, they generate poorly designed games without unique or engaging mechanics which often stems from failing to conduct effective, quantifiable usability testing.
Aeolus Software presents RealmForge, our own unique take on the roguelite genre. We aim to create a game which is built upon a modular base, allowing us to easily vary core UI and gameplay mechanics. Through the multitude of variations of our game we will produce, we will conduct extensive testing on different features and designs to determine what provides the best overall experience for our users.

G.2 Current situation

Currently, independent game studios look toward the roguelite genre due to its huge success in recent years as well as a realistic way for smaller studios to replicate the massive amounts of content larger studios are able to create. Roguelites allow for independent game developers to circumvent the limitations caused by fewer resources compared to AAA studios by prioritizing the replayability of the game through randomized runs and how different mechanics can interact with each other to create entirely new experiences.

Despite these architectural advantages, the development process of these independent game projects often have shortcomings due to relying on intuition, and poor, unstructured playtesting. Often, independant developers make critical mistakes while balancing the game, for instance, testing it only with their friends and family who provide biased feedback and do not meet characteristics of all target audiences. Games are also frequently tested too late in the development process when it is too late to make significant changes. Furthermore, developers often do not have objective metrics, relying solely on the subjective impressions received from play testers. Developers do not have reliable statistical data on what mechanics and interface elements engage the players for the longest time.

G.3 Expected benefits

The successful development and deployment of RealmForge will yield several key benefits designed to resolve the current shortcomings in independent game roguelite design and usability evaluation:

- **A Complete Core Gameplay Loop:** RealmForge will deliver a fully playable roguelite experience featuring a central hub, randomized or branched area traversal, dynamic enemy encounters, and persistent resource management that rewards players across multiple runs.
- **Empirical Usability Framework:** Provide a framework to allow developers to easily extract player data from testing to drive future development.
- **Configurable Design Flexibility:** The system will provide explicit runtime feature variation capabilities, allowing developers and researchers to toggle interface layouts, combat balancing, and mechanical behaviors on the fly without recompiling the executable.
- **Objective Data-Driven Insights:** Moving away from subjective intuition, RealmForge will equip designers with concrete statistical data to accurately evaluate player engagement, identify pacing bottlenecks, and determine which mechanical variations genuinely optimize the user experience.

G.4 Functionality overview

Functional Overview

Setting properties: The system application starts up and the user will be shown options that will allow the user to pick and change various settings during the start of the game. This will allow the user to experiment with different variants of the game which in turn alters the experience of the user with game.

Traversal: The system will display the HUD as well other UI components such as the main game screen and travel menu. This will allow the player to navigate to different areas that contain various rewards and opponents. The player will be shown the possible risks and rewards before deciding to enter a certain area.

Combat: The player will enter combat with an enemy when entering a new zone or area. The system will transition into a combat phase to which the player faces the adversary. Once in combat, a battle screen will appear as well as the player's in-game character and enemy sprite. The player will be shown various options to deal with the encounter. For example, if the player chooses to attack the enemy, an attack action is displayed followed by the enemies opposing attack.

Reward: If the player defeats the enemy, combat ends and the player will be rewarded with resources used for currency later on.

End of Run: If the player character's health points reaches zero during combat, the player will die and be returned to the main hub/starting area. The player will be able to spend resources and start the main gameplay loop again via traversal.

Spending Resources: When the player is in the main hub, the player shall be allowed to spend the resources accumulated throughout the previous journies to construct buildings and upgrades that will aid the player for the next gameplay run.

Non-Functional Overview

Ease of use: The system will display hints and instructions that will allow the player to navigate and play the game with little or no external help.

Dynamic difficulty and experience: Changing the initial properties of the game should let the player handle how difficult the game is based on formal experience. Continuously altering properties should keep the gameplay from being stale and repeatitive.

G.5 High-level usage scenarios

Use Case 001 (UC-001): Selecting Properties

1. User launches the RealmForge software.
2. User selects feature variation configurations.
3. User decides which features of the game to be changed.
4. User finishes setting up by beginning the game.

Traceability: Refined by SFR-001, SFR-002, SFR-003, SFR-004, SFR-005, SFR-018, SFR-019, SFR-006, SFR-007, SFR-010, SFR-019, SFR-018, SNFR-003, SNFR-007, SNFR-010

Use Case 002 (UC-002): Playing a full session

1. User selects to begin a new game session.
2. User enters the main game world.
3. User decides which area of the game world to go to explore based on the information given for each area.
4. User selects which area to go to and begins traveling.
5. User enters a new area and begins an encounter with an enemie/enemies (see UC-003).
6. User is successful (continue below) or fails in the encounter (see UC-005).
7. User continues traveling the game world (repeats step 2 through 6).
8. User completes the win condition for the game session (see UC-005).

Traceability: Refined by SFR-006, SFR-007, SFR-009, SFR-010, SFR-011, SFR-012, SFR-013, SFR-014, SFR-015, SFR-016, SFR-017, SFR-021, SFR-022, SFR-023, SFR-024, SFR-025, SFR-026, SFR-027,SFR-029, SFR-030, SNFR-004, SNFR-007, SNFR-009, SNFR-010

Use Case 003 (UC-003): Battling enemies

1. User enters an encounter with the enemie/enemies.
2. User selects attacks to defeat the enemie/enemies.
3. User repeats step 2 until the enemie/enemies are defeated.
4. User collects rewards for successfully completing the encounter.
5. User returns to game's world map.

Traceability: Refined by SFR-006, SFR-007, SFR-011, SFR-012, SFR-013, SFR-014, SFR-015, SFR-016, SFR-017

Use Case 004 (UC-004): Upgrading and purchasing items

1. User enters town/city/central hub.
2. User chooses and selects an available option(s) for purchasing.

3. User confirms the purchase.
4. User repeats steps 2 and 3 until the user is satisfied or resources are spent.
5. User leaves the town/city/central hub to continue the game session.

Traceability: Refined by SFR-008, SFR-025, SFR-026, SFR-027, SFR-028, SFR-029, SFR-030

Use Case 005 (UC-005): Continuing or start a new game

1. User completes a game session or fails in between (see UC-002).
2. User is given an option to return to the central hub (see UC-004) or quit the current game session (see below).
3. User selects to change features (see UC-001) or proceeds with the same configurations.
4. User begins a new game.

Traceability: Refined by SFR-018, SFR-019

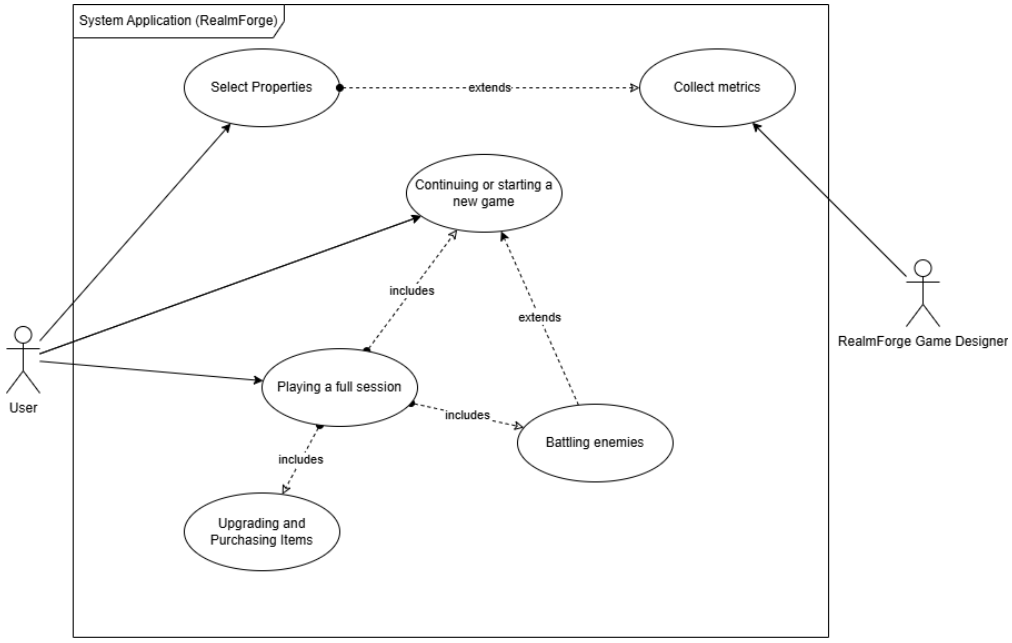


Figure 1: Use Case Diagram

G.6 Limitations and exclusions

- Force closure of the system - The system will not account for the system being force closed by the user. If this occurs, any unsaved data will be lost.
- Quality of physical input devices - The system will not account for flaws in the input device (e.g. joystick drift, input delay) other than disconnection.
- Quality of display device - The system does not ensure the reliability or connectivity of the display device used.

G.7 Stakeholders and requirements sources

The list of stakeholders, along with their relevancy to the project, are as follows:

- Capstone course facilitators: This group oversees development, and gives important advice which informs many important project decisions, particularly to do with logistics and scope.
- People who have game development experience: This group can give perspective/feedback about technical aspects of game development in a unique way from other stakeholders.
- People who play video games and are experienced with the roguelike/roguelite genre: This group would be the main demographic the game is catered to, and would have feedback which is most relevant to the experience as it pertains to the genre.
- People who play video games but are not experienced with the roguelike/roguelite genre: This group would be important to determine the approachability of the genre as a whole.
- People who have little to no experience with video games: This group would be important to the usability testing, and inform requirements to have variations in the game be more accessible.

Environment

Contents

E.1 Glossary 12
E.2 Components 15
E.3 Constraints 16
E.4 Assumptions 17
E.5 Effects 17
E.6 Invariants 18

E.1 Glossary

- **User** - The main stakeholder that is operating the system; providing inputs and receiving outputs.
- **Player** - Is another name refering the user.
- **User-Interface (UI)** - The entire set of displayed service elements the player used to navigate the system.
- **Heads-Up-Display (HUD)** - The set of interface elements displayed ontop of the actually gameplay that contains vital information helpful to the player or user.
- **Central Processing Unit (CPU)** - The central processor for a computer.
- **Streaming Single Instruction, Multiple Data Extentions 4 (SSE4.2)** - A processor instruction set used by most modern processors.
- **Advanced Micro Devices (AMD)** - An American semiconductor company that produces processors.
- **Graphics Processing Unit (GPU)** - A computer's processor for displayed graphics.
- **Open Graphics Library (OpenGL)** - An application programming interface used to render computer graphics.
- **NVIDIA Corporation** - An American technology company that produces graphics processors.

- **Graphics Core Next (GCN)** - A series of graphics processor architectures that is used in AMD graphics processors.
- **Random Access Memory (RAM)** - Short-term memory used by a computer's processor.
- **Frames per Second (FPS)** - The number of graphical frames displayed by a computer program per second.
- **Personal Computer (PC)** - A computer owned by individuals, for personal use.
- **Verification and Validation (VnV)** - The process of ensuring that a design satisfies its requirements, and its requirements are aligned with the design's purpose (generally).
- **Feature-Oriented Domain Analysis (FODA)** - A domain analysis method, focusing on features of a design and their variations.
- **Proof of Concept (POC)** - An initial version of a design, intended to demonstrate the possibility of its implementation without being a full prototype.
- **Roguelite** - A video game genre featuring levels with randomized elements that change every playthrough and permanent death, meaning dying resets the current run. Players retain permanent upgrades unlocked between runs that make their character stronger over time.
- **Sprite** - An in-game 2-dimensional picture representing an object in the 2D-world.
- **GDUnit4** - A Godot module that can be added to any base godot project. The module allows for the creation of unit test cases.
- **Inventory** - The set of resources, items, and currency store within code and available to access for the player
- **GDLint** - A Godot module that can be added to any base godot project. The module is a linting tool to help will proper coding styles and errors for the Godot language.
- **Integrated Development Environment** - A set of software applications and tools within a programmer's arsenal to help develop software code efficiently.
- **Operation System** - A collection of software that manages a computer's hardware and software applications by handling allocation of resources.
- **LaTeX** - LaTeX is a high-quality typesetting system; it includes features designed for the production of technical and scientific documentation.
- **Continuous Integration / Continuous Development** - An iterative approach to software development where updates are pushed in small increments and re-evaluated for further development

- **Copper** - A metal typically used for forging armor and sometimes used as a bartering tool.
- **Skeleton** - A human structure composed of bones. Commonly used in games to as a moving enemy to obstructs the player.

E.2 Components

Included below is a System Context Diagram, outlining at a high level how the system will interact with external components:

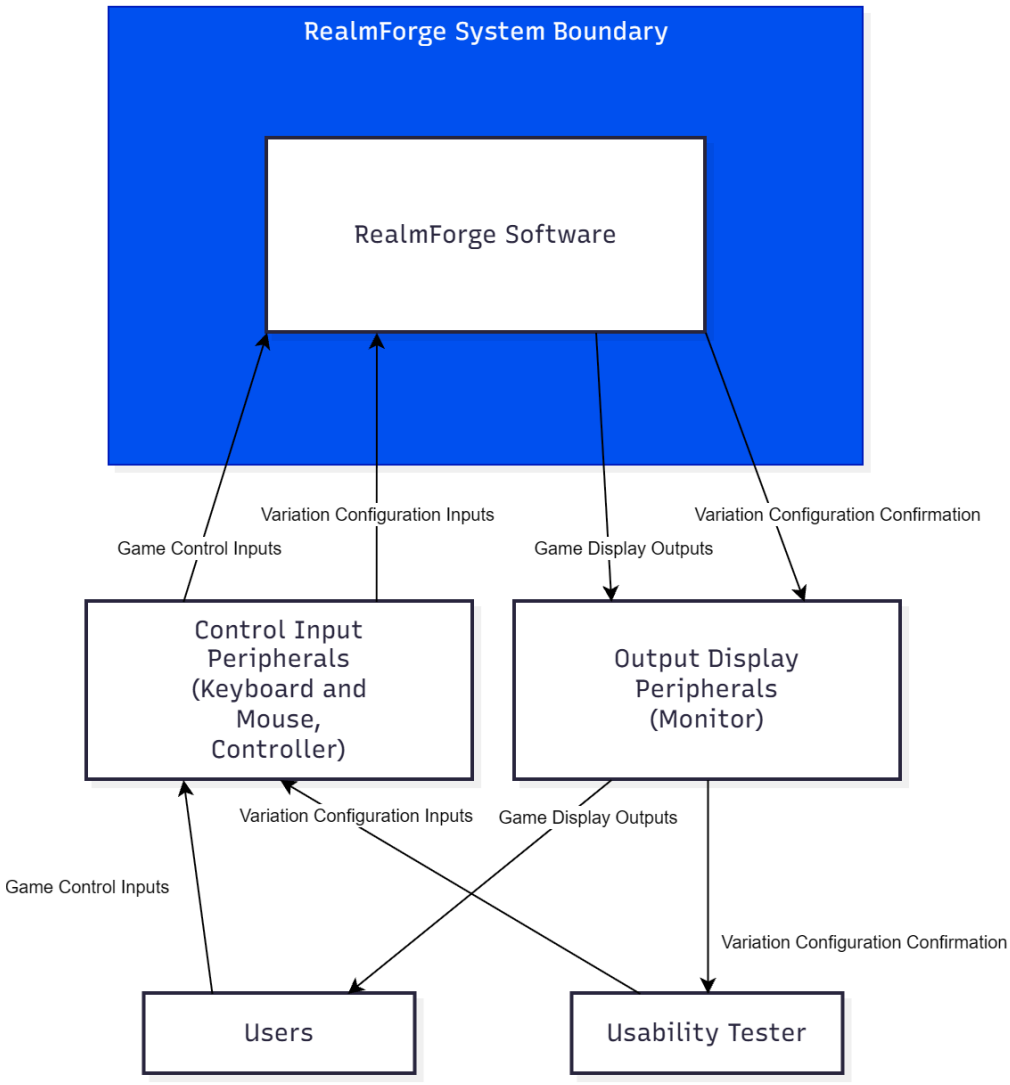


Figure 2: RealmForge System Context Diagram

The RealmForge software (also denoted as the system) will require the use of the following external components to operate successfully:

- **Control Input Peripherals:** Devices used to provide input to the system. This could include a keyboard and mouse, or a controller used to provide input. The system will require inputs from these devices to affect the state of the video game it

executes (during gameplay), and to configure the system's feature variation points during testing.

- **Output Display Peripherals:** A device used to display outputs provided by the system. This could be fulfilled by a monitor, or another display device (such as a television). The system will require this device to display graphical outputs during gameplay (video game graphics), as well as confirmation of feature variation configurations (variation config. menu graphics).

E.3 Constraints

ENV-001: The system shall be compatible with user machines running Windows 10 or 11 (64-bit) operating systems that meet the following recommended specifications:

- **CPU:** x86_64 CPU with SSE4.2 support, with 4 physical cores or more. Example: Intel Core i5-6600K, AMD Ryzen 5 1600.
- **GPU:** Dedicated graphics GPU with full OpenGL 4.6 support. Example: NVIDIA GeForce GTX 650, AMD Radeon HD 7750 (GCN 1.0).
- **RAM:** A minimum of 8 GB of RAM.

Rationale: This environment constraint is imposed to focus development of RealmForge on the most commonly used operating system for PC video games, Windows. The system may be extended to support other operating systems in the future if possible. The recommended specifications are included to ensure user machines can fully execute the system, accompanying any future decisions to extend its capabilities.

Fit Criterion: The system can be compiled and executed on machines running Windows 10 or 11 (64-bit) operating systems, meeting the recommended specifications outlined above, at a stable 60 fps.

Traceability: N/A

ENV-002: The system shall provide user input capabilities for keyboard and mouse peripherals, as well as common controllers (PlayStation, Xbox).

Rationale: The system must accommodate the use of various input methods (keyboard and mouse, different controllers) to facilitate usability studies incorporating these varying input methods.

Fit Criterion: The system can receive all game inputs required for its successful operation from the following input methods: USB Keyboard and Mouse, Xbox Series X/S, Xbox One, or Xbox 360 controllers, and PlayStation DualSense or DualShock 4 controllers.

Traceability: Satisfied by SFR-001, SFR-003

ENV-003: The system shall provide at least one explicit feature variation capability, that can be selected without recompiling the system's executable file.

Rationale: This constraint is imposed by the purpose of the development project (outlined in the Problem Statement document). The system must provide explicit feature variation capabilities to facilitate usability testing that will focus on the effectiveness of different variations.

Fit Criterion: The system provides at least one explicit feature variation capability, that can be selected and managed at run time, while the system is executing.

Traceability: N/A

E.4 Assumptions

The following assumptions describe the environment in which RealmForge is expected to operate. These include the user's device setup and external components that are outside the system's control.

Display Device

It is assumed that players have access to a properly connected and functioning display device that is capable of displaying RealmForge's graphics.

Input Devices

It is assumed that players have access to properly connected and functioning input devices compatible with their system, such as a keyboard and mouse or a supported controller, to interact with the game.

Operating System and Driver Support

It is assumed that the user's operating system has the necessary drivers installed and configured to support the input and display devices used by RealmForge.

Application Permissions

It is assumed that users have permission to install and run RealmForge on their devices, as well as the necessary permissions to access local files used to save game progress.

Available System Resources

It is assumed that the user's device has enough available memory and processing power to run RealmForge without significant interference from other applications running at the same time.

E.5 Effects

The following effects describe how RealmForge is expected to affect the user's environment while the game is running. These include the use of system resources, interactions with external devices, and changes to local storage.

System Resource Usage
While RealmForge is running, the user's CPU, GPU, and RAM will be used to support gameplay and game performance. The amount of system resources used may vary depending on the player's actions and other applications running in parallel.

Display Output
The game will be presented on the user's display through graphical content. The graphical content will update based on the player's actions and progress throughout the game.

Audio Output
RealmForge will produce sound during gameplay through the user's speakers or headphones, depending on their selected audio output device.

Input Devices
RealmForge will receive input data from supported devices connected to the system. Inputs from the keyboard and mouse will be read and reflected during gameplay, allowing the player to interact with the game.

Local Storage
Installing RealmForge will require the application to use local system storage for executable files and other necessary game files. User progress during gameplay will also be saved locally, requiring additional storage space on the user's device.

E.6 Invariants

The following properties of the environment must be preserved during RealmForge's operation, including local files, operating system settings, input devices, user privacy, and audio and display settings outside of gameplay.

Files & Storage
RealmForge must not modify or delete any existing files unrelated to its own installation and game data. Similarly, uninstalling the game should not remove any local files unrelated to RealmForge.

Operating System Settings
The overall operating system configuration must remain unchanged by RealmForge. System settings and security configurations must not be permanently modified during gameplay.

Input Device Functionality
Input device functionality must remain unchanged outside of RealmForge gameplay. Keyboard and mouse functionality must remain available to the user, and any game-specific changes affecting these devices must be restored once the game exits.

User Data & Privacy
User progress in RealmForge must be preserved between game sessions. Private user data must not be accessed or collected beyond what is required for gameplay and disclosed to the user.

Audio & Display Settings
Overall audio and display configurations must remain unchanged outside of the game. Any changes made to the system's display or audio settings during gameplay must be restored once the game session ends. RealmForge must not modify the audio settings of other applications running in parallel.

System

Contents

S.1 Components **20**
S.2 Functionality **21**
S.3 Interfaces **31**
S.4 Detailed usage scenarios **31**
S.5 Prioritization **39**
S.6 Verification and acceptance criteria **39**

S.1 Components

The system can be broken down into the following core components:

- **Physical Controls**: This component handles input from the player. This can be achieved through a computer keyboard or a standard game controller. The input enabled by this component allows players to interact with other components of the system and is typically constrained/enforced through the UI.
- **User Interface (UI)**: This component allows the player to interact with the game (using their preferred interaction method). This is shown through various menus, buttons and other visual prompts and ensures the game is intuitive to navigate while constraining the player to certain intended methods of interaction.
- **Game Settings**: This component contains various selections that allow the player to change aspects of the game. This allows the player to customize how they interact with the game in ways such as gameplay, audio, and visual changes. These changes will persist as stored variables to signify current settings within the rest of the system.
- **Combat System**: This component contains an integral piece of the system, as selected using the map system. This is the main way the player will interact with the game. Combat encounters will be part of a game scene, which when started is initialized using player and enemy properties, possible actions, current game state and the condition(s) needed to win.

- **Map System**: This component contains the main means of traversal for the player, creating a linear progression (with some branching and converging) through the various game scenes (encounters). This component gives the player a sense of progression and a goal to work towards.
- **Central Hub**: This component contains the main source of long-term progression data in a game world 'hub area'. Various relevant game statistics and data related to the central hub can be tracked in the game's save file and updated upon return to the hub.

S.2 Functionality

Physical Controls:

Functional Requirements:

SFR-001: The controller button-bindings shall have a keyboard and mouse equivalent.

Rationale: The user must have full access to all functionality of the game with either a controller or keyboard and mouse. This helps if the user is not experienced or comfortable with a controller option.

Fit Criterion: The directional inputs of the d-pad from the controller has the same input mappings as the left, right, up, down on the keyboard. The A, B, X, Y, L and R buttons on XBOX controllers also have a keyboard equivalent functionality mapped into certain keys.

Traceability: Satisfies ENV-002, Refines UC-001, US-001, Satisfied by SFR-002, SFR-003, SFR-004,

SFR-002: The game shall explain what each button on the connected controller can do for the user. The same goes for the keyboard and mouse (see SFR-001).

Rationale: Identifying the actions of what each button will do will prevent the player from feeling lost or perform an action that was not intended.

Fit Criterion: The game displays a diagram of a controller that marks all the actions with the correlated button.

Traceability: Satisfies SFR-001, Refines UC-001, US-001,

SFR-003: The game shall default to the keyboard and mouse as the main source of input handling.

Rationale: A keyboard and mouse are more likely to be connected since the software will be designed to run on a computer.

Fit Criterion: During the launch of the software, the software will accept keyboard key inputs, and the user will have to manually switch to the controller.

Traceability: Satisfies ENV-002, SFR-001, Refines UC-001, US-001 Satisfied by SFR-005,

SFR-004: The game shall give the option for the user to switch between using a controller or keyboard and mouse as inputs.

Rationale: This gives the player and the game knowledge of how the inputs are being handled, whether via keys or controller buttons.

Fit Criterion: During the start screen, there is an option to change from the default keyboard, and mouse or connected controller.

Traceability: Satisfies ENV-002, SFR-001, SFR-003, Refines UC-001, US-001 Satisfied by SFR-005,

SFR-005: The game shall handle only either a keyboard and mouse or a controller inputs at a single instance, not both at the same time.

Rationale: If a controller and keyboard are connected, this prevents the user from accidentally pressing inputs that were not intended.

Fit Criterion: During the launch of the software, if a controller is connected, the game will not accept any action from it unless switched to.

Traceability: Satisfies SFR-003, SFR-004, Refines UC-001, US-001

Game Settings:

Functional Requirements:

SFR-006: The game shall have a configuration that changes the player's starting health.

Rationale: This is a difficulty setting to test how far a user can last on average before failing and restarting a run. For example, a user with minimum gaming experience will typically have a better time with the game if the user can last a little longer. Dialing the health will also limit how far a player goes on their initial runs, thus preventing them from seeing too much of the game early on. This configuration helps balance the player's overall experience with the pace at which the game's content is consumed.

Fit Criterion: The user can change the starting health from a minimum of 50 to 150.

Traceability: Refines UC-001, UC-002, UC-003, US-001, US-002, US-003, Satisfied by SFR-009, SNFR-003, SNFR-008

SFR-007: The game shall have a configuration that changes the average amount of resources gained after an encounter.

Rationale: This is a pacing mechanic for the game. The user should not be able to purchase all the upgrades from the central hub with only a short few runs; otherwise, the initial difficulty of the game gets trivialized by the added upgrades; not by the player's own skill. Having this configuration will also help distinguish how long the average player feels about the overall pace of the game; different users have varying tolerances between the feeling of progression and repetition.

Fit Criterion: The user can change the average number of resources dropped between 10 and 20 for enemy encounters.

Traceability: Refines UC-001, UC-002, UC-003, US-001, US-002, US-003, Satisfied by SFR-008, SFR-009, SNFR-008

SFR-008: The game shall have a configuration that changes the cost of each purchasable upgrade.

Rationale: Changing the purchase cost is slightly different from SFR-007, because it changes how players perceive the purchase progression. A player will gauge how far they must go before purchasing the next upgrade. If the cost is too high, then this discourages the player from buying things. If the cost is too low, then this runs the risk of completing all purchases too fast; there needs to be a balance for the cost.

Fit Criterion: The user can change the additional cost of each purchasable upgrade from zero to 50.

Traceability: Satisfies SFR-007, Refines UC-004, US-004, Satisfied by SFR-009, SNFR-003, SNFR-008

SFR-009: The game shall prevent the user from changing the game's configurations until starting a new game.

Rationale: Altering the game's parameters while playing trivializes the idea of game variants entirely. If a user changes a configuration, then it could be because of inconvenience, and thus ruins the idea of difficulty. This can lead to an unstable game experience. This also prevents bugs due to changing variables. For example, changing the initial starting health while the game states the current health is zero.

Fit Criterion: The game will not include an option to allow the user to stop the game and immediately change a setting such as initial starting health.

Traceability: Satisfies SFR-008, SFR-007, SFR-006, Refines UC-002, UC-002, US-001, US-002, Satisfied by SFR-010

SFR-010: The game shall give a prompt for confirmation once the user has finished setting up the game's configurations.

Rationale: The game's configurations cannot be changed (see SFR-009). This gives the user a contractual obligation to finish the game whilst under the rules laid out.

Fit Criterion: When a user exits from the configuration's menu, the game will display a prompt showing the question "Is the configuration complete" followed by yes-or-no text buttons.

Traceability: Satisfies SFR-009, Refines UC-001, UC-002, US-001, US-002

Combat System:

Functional Requirements:

SFR-011: The game shall display a visual for both the player and enemies' combat representation.

Rationale: This is needed to alert the user for when a combat encounter has occurred and gives a sense of the borders and positioning during combat.

Fit Criterion: The game displays an image for the player's character on the bottom-left screen and the enemies' images in the top-right screen.

Traceability: Refines UC-002, UC-003, US-002, US-003, Satisfied by SFR-012, SFR-014, SFR-017, SFR-020, SFR-021

SFR-012: The game shall display the player and the enemies' respective health points during the combat screen.

Rationale: This is needed to indicate the urgency of the situation during combat; whether the player is losing or winning.

Fit Criterion: The game displays health bars under their player and enemies' images (see SFR-011).

Traceability: Satisfies SFR-011, Refines UC-002, UC-003, US-002, US-003, Satisfied by SFR-013, SFR-015

SFR-013: The game shall display an indication of success if the enemy's health goes to zero or failure if the player's health goes to zero.

Rationale: This is needed in order to know when combat has concluded.

Fit Criterion: The game displays a 'player wins' text when the enemies' health is reduced to zero. The game displays a 'player has failed' text when the player' health is reduced to zero.

Traceability: Satisfies SFR-012, Refines UC-002, UC-003, US-002, US-003, Satisfied by SFR-021, SFR-019, SFR-020

SFR-014: The game shall display an attack visual whenever the player presses an attack option. Similar, the game shall display a visual for the enemy's opposing attack.

Rationale: The combat needs to be telegraphed/presented to the player in some manner.

Fit Criterion: The game displays the in-game characters weapon swinging to indicate the attack.

Traceability: Satisfies SFR-011, Refines UC-002, UC-003, US-002, US-003, Satisfied by SFR-016, SNFR-002, SNFR-005

SFR-015: The game shall indicate when the player or enemies' health is changed (decreased or increased).

Rationale: The player must know whether an attack done by the player or enemy was effective or not, otherwise, there is no sense of progress of combat.

Fit Criterion: When the player attacks an enemy, the enemy's health bar updates accordingly.

Traceability: Satisfies SFR-012, Refines UC-002, UC-003, US-002, US-003

SFR-016: The game shall display at one or more combat options for the player.

Rationale: This is mandatory for progression during combat. Otherwise, the player will not be able to do anything.

Fit Criterion: The game displays UI elements labeled 'attack' and 'defend' during combat.

Traceability: Satisfies SFR-014, Refines UC-002, UC-003, US-002, US-003

SFR-017: The player shall have the possibility to encounter more than one enemy at a time.

Rationale: This improves combat experience, because the player must now prioritize and deal with a certain enemy first over the others.

Fit Criterion: The game displays two skeleton enemies. Enemy type is an example only.

Traceability: Satisfies SFR-011, UC-002, UC-003, US-002, US-003

User Interface System:

Functional Requirements:

SFR-018: The game shall first launch and display a screen with UI component options for starting a new game, configurations, and exiting the game.

Rationale: This is mandatory because these are the actions the user faces first before everything else.

Fit Criterion: The game displays 3 text-buttons with text saying 'start game, 'settings', and 'exit'.

Traceability: Refines UC-001, UC-005, US-001, US-004

SFR-019: The game shall display a screen with options to restart the game or continue a game session after the player has failed or completed the win condition of the run.

Rationale: This is the end point for the main loop of the game. The game cannot proceed properly without this key user decision.

Fit Criterion: The game displays 2 text-buttons with text saying 'restart game, and 'continue game'.

Traceability: Satisfies SFR-013, Refines UC-001, UC-005, US-001, US-004

Map System:

Functional Requirements:

SFR-020: The game shall show when the player is currently in the overworld of the game.

Rationale: The player needs to know which situation or state the game is in, whether it is combat or exploration.

Fit Criterion: The game displays a map screen along with multiple locations with pathways linking towards each location

Traceability: Satisfies SFR-011, SFR-013, Refines UC-002, US-002

SFR-021: The game shall give at least one or more travel options.

Rationale: The player needs to have the freedom to choose where to go next to give the game variety and not feel like it is on a linear path.

Fit Criterion: The game displays UI elements titled 'Bahamas', 'Volcano', and 'Island' followed by an option to travel. The titles are examples only.

Traceability: Satisfies SFR-011, Refines UC-002, US-002, Satisfied by SFR-022

SFR-022: The game shall show a prompt to the player for confirmation of travel.

Rationale: This give the player reaffirmed knowledge and an update on the travel location.

Fit Criterion: The game displays a prompt after the player has chosen a travel destination. The prompt displays "Are you sure" with yes-or-no options.

Traceability: Satisfies SFR-021, Refines UC-002, US-002, Satisfies by SFR-023

SFR-023: The game shall indicate to the player where the player is traveling to.

Rationale: This is mandatory so that the player is not confused on if they travelled, and where they travelled to.

Fit Criterion: The game displays an image of the player character moving towards a travel marker beside the next location.

Traceability: Satisfies SFR-021, SFR-022, Refines UC-002, US-002, Satisfied by SNFR-004

SFR-024: The game shall show the name of the area.

Rationale: The player should know how far in the game they have reached toward the end.

Fit Criterion: The game shows a label of the area the player character is currently at the top right of the screen.

Traceability: Refines UC-002, US-002

Central Hub:

Functional Requirements:

SFR-025: Permanent progression shall be characterized by the player keeping some progression after they lose; either currency, or materials the player discovered in each attempt through combat or both.

Rationale: The player should feel some sense of improvement or accomplishment during the gameplay, otherwise they may feel demotivated and would not feel the need to continue.

Fit Criterion: The game keeps track of the copper collected after the player has failed a run.

Traceability: Refines UC-002, UC-004, US-002, US-004

SFR-026: During gameplay, the player shall gain resources sent from the central hub, changing the gameplay experience for each run. This progression will not persist after the player loses and starts a new game.

Rationale: This is a vital feature the game uses to help improve the odds of the player going further during a game session.

Fit Criterion: The game transfers magic points to the player each combat instance. This is an example only and more possibilities will be available.

Traceability: Refines UC-002, UC-004, US-002, US-004

SFR-027: The game shall give one or more options for the player to purchase items.

Rationale: The is mandatory since there is otherwise no way for the player to become stronger or get any help during the game.

Fit Criterion: The game displays images of the items/buildings/weapons purchasable as well as a 'buy' option underneath.

Traceability: Refines UC-002, UC-004, US-002, US-004, Satisfied by SFR-028, SFR-029, SFR-030, SNFR-001

SFR-028: The game shall display the resource cost for each item to be purchased.

Rationale: This is needed, otherwise the player will not be aware of how to allocate the resources gathered during each run.

Fit Criterion: The central hub shows a value for the in-game currency needed next to the item's image.

Traceability: Satisfies SFR-027, UC-002, UC-004, US-002, US-004, Satisfied by SFR-029, SFR-030

SFR-029: The game shall describe the effect of the item in some way to the player.

Rationale: The player needs to know the effect before purchasing to make an educated guess on which upgrade/enhancement to prioritize purchasing.

Fit Criterion: The game displays text for a detailed description for the use case of the purchasable.

Traceability: Satisfies SFR-028, SFR-027, Refines UC-002, UC-004, US-002, US-004

Undesired Event Handling:

SFR-030: The game shall reject the player in some way from purchasing the item if the player does not have the needed currency or materials required for purchased.

Rationale: Stops the player from being able to buy equipment without meeting the value prescribed to the item.

Fit Criterion: The game displays a text box saying 'Sorry, not enough materials' after the player attempts to buy the item.

Traceability: Satisfies SFR-028, SFR-027, Refines UC-002, UC-004, US-002, US-004

Non-Functional Requirements:

Look and Feel Requirements:

SNFR-001: The game shall give a variety of purchase options that differ in the ways to improve the next game session.

Rationale: This gives the player the ability to try and test out all the different purchasable upgrades in the central hub in order to weigh their worth; it compels the user to experiment.

Fit Criterion: The game provides an upgrade to increase the amount of attacks a player can deal, being able to travel faster, or being able to collect more resources after combat. These are examples and more options are possible.

Traceability: Satisfies SFR-027

SNFR-002: The player shall encounter different kinds of enemies varying in different ways such as attack types, health, armor, etc.

Rationale: The player doesn't want to feel bored fighting the same enemy over again. This feels repetitive and discourages the player from continuing the game.

Fit Criterion: The player can encounter an enemy that can attack physically as well as an enemy that poisons the player (slowly decrease health over time). The player can encounter an enemy that has increased armor (damage done to it is less than normal), and an enemy with more health (health bar is longer). These are examples and more options are possible.

Traceability: Satisfies SFR-014

SNFR-003: The game shall have configuration options that alter the experience, such as difficulty and pace of the game.

Rationale: The game shall have adjustable variables that will help tune the game for testing to better experience overall.

Fit Criterion: The game has a configuration option that changes how fast the combat is between the player and enemy.

Traceability: Satisfies SFR-006, SFR-008, Refines UC-001, US-001

SNFR-004: The game shall showcase a variety of different travel destinations that are visually different.

Rationale: This helps gives the player external knowledge and hints of the risks the player faces when heading to a certain location.

Fit Criterion: The game displays images for locations showing a nice island, a volcano, a desert and grassy area. These are examples and more options are possible.

Traceability: Satisfies SFR-021, Refines UC-002, US-002

SNFR-005: The player shall have offensive and defensive options when facing an encounter.

Rationale: This keeps the player engaged, otherwise, the player is repeating the same option of attacking with little to no change.

Fit Criterion: The game gives the player the options to charge an attack (double damage next turn attack), attack normally, or defend with a shield. These are examples and more options are possible.

Traceability: Satisfies SFR-014, Satisfied by SNFR-006

SNFR-006: The game shall telegraph the type of enemy encounter to the player.

Rationale: The player should know how to handle each type of enemy.

Fit Criterion: The game displays an enemy with a sword and another enemy with a shield; whenever the player attacks the shielded enemy, it takes less damage than the sword enemy. These are examples and more options are possible.

Traceability: Satisfies SNFR-002

Performance Requirements:

SNFR-007: The inputs for the controller being used shall have immediate feedback response for the game.

Rationale: Any major input delays will make the game feel slow to respond to any player feedback.

Fit Criterion: The game response time in handling the players input is less than half a second.

Traceability: Satisfies UC-001, UC-002

Normal Operation Requirements:

SNFR-008: The game shall give a description to all the game selection properties.

Rationale: This helps the player in deciding which options they want to tackle.

Fit Criterion: The game displays a tool-tip text when the user selects a configuration for change.

Traceability: Satisfies SFR-006, SFR-007, SFR-008

SNFR-009: The game shall have valid transitions to indicate the game changing state

Rationale: This prevents the player from being disoriented or confused whether they are in combat or traveling.

Fit Criterion: The game fades in and out (turn complete black and reappear) whenever the player enters combat.

Traceability: Satisfies UC-002, US-002

Likely to change or Out of current scope Requirements:

SNFR-010: The game shall provide audio cues for when the player does an action.

Rationale: This helps gives the player feedback response that is not visual.

Fit Criterion: The game creates a sound effect whenever the player confirms a prompt.

Traceability: Satisfies UC-001, UC-002, US-001, US-002

S.3 Interfaces

Comment: *How the system makes the functionality of S.2 available to the rest of the world, particularly user interfaces and program interfaces (APIs).*

S.4 Detailed usage scenarios

US-001

User Scenario Name: System Feature Variation Configuration

Context: In this scenario, a user of RealmForge (also denoted 'the system') intends to configure the system's feature variation capabilities, in the context of a usability study (also called a 'playtest'). They will do so through the assistance of the RealmForge developers conducting the study/test, if needed.

Primary Actor: RealmForge User

Supporting Actor(s): RealmForge Developer

Precondition:

- The RealmForge software is installed and compiled on the user's machine used for testing
- The user has initially connected a keyboard and a mouse to their machine to provide input to the system
- A RealmForge Developer is present (virtually or in-person) to facilitate testing

Minimal Postcondition:

- The system returns to its main menu from the feature variation configuration menu
- The system remains in a valid system state (does not crash, the program does not close)

Success Postcondition:

- The system returns to its main menu from the feature variation configuration menu
- The system remains in a valid system state (does not crash, the program does not close)
- The system configures RealmForge's feature variations according to the user's selections, and provides feedback to the user regarding the configurations implemented
- The user is able to launch a new game session, implementing their selected configurations

Trigger: User launches the RealmForge software, and prompts the system to configure feature variations from its main menu

Main Success Scenario:

1. System displays feature variation configuration menu
2. User opts to configure input method
3. System displays input device options (keyboard and mouse, controller)
4. User selects controller option and connects a controller to their machine
5. System recognizes input and begins to respond to controller input
6. System returns to feature variation configuration menu
7. User opts to configure gameplay features
8. System displays possible gameplay feature variations
9. User selects unique gameplay feature configuration, in accordance with playtest plan
10. System configures RealmForge game build with the user's selections
11. System returns to feature variation configuration menu
12. User opts to exit to main menu
13. User opts to start a game session

14. System launches a new game session that implements the user's configurations
15. System displays the initial game state's graphics to the user

Secondary Scenarios:

- Alternative Scenario:
  - 2.1-6.1: User intends to test RealmForge gameplay using keyboard and mouse input, and does not opt to switch their input method using the system
- Alternative Scenario:
  - 9.2: User is unsure of which feature variation configuration to select, and consults the RealmForge developer; developer provides the next configuration to select as part of testing, User selects the configuration using the system
- Exceptional Scenario:
  - 5.3: System does not recognize input method selected by the user; RealmForge developer assists the user with troubleshooting the issue, user successfully connects the input device, and the system recognizes its input and begins to use the device as a main input

Traceability: Refined by SFR-001, SFR-002, SFR-003, SFR-004, SFR-005, SFR-018, SFR-019, SFR-006, SFR-007, SFR-010, SFR-019, SFR-018, SNFR-003, SNFR-007, SNFR-010

US-002

User Scenario Name: Full Gameplay Session

Context: In this scenario, a user of RealmForge intends to play through the game. They begin a new game session in a new save file, travel through the game world, complete enemy encounters, and reach the end of the game (if successful).

Primary Actor: RealmForge User

Supporting Actor(s): N/A

Precondition:

- The RealmForge software is installed and compiled on the user's machine
- The user has successfully connected a valid input device to their machine to provide input to the system
- The RealmForge software has been correctly configured with the appropriate feature variations for this gameplay session (or is configured in its final build, if this gameplay session occurs after the full release of the software)

Minimal Postcondition:

- The system remains in a valid system state (does not crash, the program does not close)
- The user's new save data is successfully stored without corruption

Success Postcondition:

- The system remains in a valid system state (does not crash, the program does not close)
- The user's new save data is successfully stored without corruption
- The user is able to fully traverse through the game world, complete enemy encounters, and reach the end of the game (if they are successful), or return to the beginning (if they are unsuccessful)

Trigger: User launches the RealmForge software, and prompts the system to begin a new game in a new save file (from the software's main menu)

Main Success Scenario:

1. System prompts the user to enter a name for the save file
2. User enters a name for their save file
3. System initializes the game, and displays the initial state of the game world to the user
4. User decides which valid game world area they will progress to next, and selects this decision via the system
5. System displays the user's player character moving to the next game world area, and displays the game entering the enemy encounter associated with the area
6. User plays through the encounter (see US-003)
7. System displays game state to user during the progression of the encounter, and displays results; the user is successful
8. System displays rewards given to the user due to their success
9. System returns game state to the overall game world, and displays this to the user
10. User repeats steps 4-9 while progressing through the game (through the use of the system)
11. User opts to progress to the final game world area

12. User repeats steps 5-7 while playing through the final enemy encounter, and is successful
13. System displays to user that they are successful, and have completed the game
14. System returns to its main menu
15. System displays the following options to the user: Begin a new game using a new save file, or play the game again (continue, by returning to the Central Hub discussed in US-004) in their original save file

Secondary Scenarios:

- Alternative Scenario:
  - 6.1-15.1: User opts to save and quit the game during an encounter; System notifies user that they will lose their current encounter progress, and saves the game state at the point before the user entered the current encounter to the user's save file
- Alternative Scenario:
  - 10.2-15.2: User opts to save and quit the game from the overall game world; System saves the current game state to the user's save file, and returns to the main menu
- Alternative Scenario:
  - 6.3-15.3: User fails during an enemy encounter; System notifies the user that they have failed, returns the game state to the beginning of the game, and presents the Central Hub menu to the user (see US-004)
- Exceptional Scenario:
  - 4.4: User attempts to progress to new game world area that they have not unlocked (it is not available yet due to their current progress); System notifies the user that their selection is not yet available, and suggests alternatives

Traceability: Refined by SFR-006, SFR-007, SFR-009, SFR-010, SFR-011, SFR-012, SFR-013, SFR-014, SFR-015, SFR-016, SFR-017, SFR-021, SFR-022, SFR-023, SFR-024, SFR-025, SFR-026, SFR-027,SFR-029, SFR-030, SNFR-004, SNFR-007, SNFR-009, SNFR-010

US-003

User Scenario Name: Enemy Encounter Game Event

Context: In this scenario, a user of RealmForge enters an enemy encounter while playing through the game, and they intend to select various game actions to progress through the encounter and defeat the included enemies.

Primary Actor: RealmForge User

Supporting Actor(s): N/A

Precondition:

- The RealmForge software is installed and compiled on the user's machine
- The user has successfully connected a valid input device to their machine to provide input to the system
- The RealmForge software has been correctly configured with the appropriate feature variations for this gameplay session (or is configured in its final build, if this gameplay session occurs after the full release of the software)
- The user has started a playthrough of the RealmForge game in a save file, and has reached the point of an enemy encounter

Minimal Postcondition:

- The system remains in a valid system state (does not crash, the program does not close)
- The system exits the enemy encounter game event

Success Postcondition:

- The system remains in a valid system state (does not crash, the program does not close)
- The system exits the enemy encounter game event
- The user is able to select actions available to them during the encounter, battle enemies using these actions, and complete the encounter and receive rewards if they are successful
- If the user is not successful, they are returned to the beginning of the game

Trigger: User opts to progress to a new game world area while playing RealmForge, and enters the enemy encounter associated with the area

Main Success Scenario:

1. System displays the initial enemy encounter game state to the user, including the enemies they must face (in this scenario one), and their player character

2. User selects a valid action to take to progress through the encounter
3. System updates the game state, the enemy's attributes, and the player character's attributes following the action, and displays the outcome to the user
4. System updates the game state with the enemy attacking the user's player character, and displays the outcome to the user
5. User and System repeat steps 2-4 as the encounter progresses
6. System updates the game state and the enemy's attributes following an attack action selected by the user; the enemy has been defeated
7. System displays that the enemy has been defeated to the user, and exits the encounter since the user has defeated all enemies
8. System updates the game state with the rewards given to the user for their success, and displays these rewards to the user

Secondary Scenarios:

- Alternative Scenario:
  - 6.1-8.1: User is defeated by an enemy attack; System notifies the user that they have failed, returns the game state to the beginning of the game, and presents the Central Hub menu to the user (see US-004)

Traceability: Refined by SFR-006, SFR-007, SFR-011, SFR-012, SFR-013, SFR-014, SFR-015, SFR-016, SFR-017

US-004

User Scenario Name: Post Gameplay Run Events

Context: In this scenario, a RealmForge user has returned to the beginning of the game, either through being defeated at some stage, or completing the game and selecting to play the game again in the same save file. At this point, they intend to upgrade their in-game abilities through upgrades of the game's Central Hub, and begin a new game "run".

Primary Actor: RealmForge User

Supporting Actor(s): N/A

Precondition:

- The RealmForge software is installed and compiled on the user's machine

- The user has successfully connected a valid input device to their machine to provide input to the system
- The RealmForge software has been correctly configured with the appropriate feature variations for this gameplay session (or is configured in its final build, if this gameplay session occurs after the full release of the software)
- The user has started a playthrough of the RealmForge game in a save file, and has either been defeated and failed a game run, or completed the game and selected to play again in the same save file

Minimal Postcondition:

- The system remains in a valid system state (does not crash, the program does not close)

Success Postcondition:

- The system remains in a valid system state (does not crash, the program does not close)
- The user is able to upgrade the Central Hub and begin a new game run, or quit the game, depending on their selections

Trigger: User opts to continue playing in their current save file

Main Success Scenario:

1. System returns the game state to the beginning of the overall game world, and initializes the Central Hub menu
2. System displays the Central Hub menu options to the user
3. User selects to upgrade a Central Hub attribute using in-game currency rewarded from previous enemy encounters
4. System displays confirmation message of Central Hub upgrade purchase
5. User confirms upgrade purchase
6. User repeats steps 3-5 via the system until they have purchased each upgrade they require, or do not have enough currency to complete their purchases
7. User selects to begin a new game run and leave the Central Hub menu
8. System initializes the game world, and displays the initial game state to the user (see US-002 from step 3 onwards)

Secondary Scenarios:

- Alternative Scenario:
  - 1.1-8.1: User opts to quit to the main game menu; System exits Central Hub menu and saves game state to save file, and returns to main menu
- Alternative Scenario:
  - 1.2-8.2: User opts to quit to the main game menu (following the alternative scenario above), and selects to reconfigure system feature variations (in a testing context); System opens and displays feature variation configuration menu and displays it to the user (see US-001)
- Exceptional Scenario:
  - 3.3: User opts to purchase Central Hub attribute upgrade that they cannot currently afford; System displays a message to the user notifying them that they do not have enough currency to purchase the upgrade, and returns to the main upgrade menu

Traceability: Refined by SFR-008, SFR-025, SFR-026, SFR-027, SFR-028, SFR-029, SFR-030, SFR-018, SFR-019

S.5 Prioritization

Comment: *Classification of the behaviors, interfaces and scenarios (S.2, S.3 and S.4) by their degree of criticality.*

S.6 Verification and acceptance criteria

Acceptance of the RealmForge system will be verified with multiple methods of testing and analysis to ensure all functional, and non-functional requirements are satisfied:

- **Unit Testing:** Individual components, scripts, and helper functions will be verified using automated unit test suites. This level will be met if all unit test assertions are passed.
- **Integration Testing:** Interactions between components of our system will be tested to ensure that data is properly passed between them with zero loss. This level will be met if all components can safely interact with each other and still perform all functions.
- **System Testing:** The integrated software will be tested against environmental constraints. This level is met when the system successfully compiles, launches, and executes with no unhandled exceptions, and maintains a stable framerate.
- **User Acceptance Testing (UAT):** The system's behaviors will be validated against the user scenarios via structured playtesting and usability studies. This

level will be met when users are observed to interact with the system as desired and user reviews are in accordance with our requirements/goals.

- **Static Analysis and Linting:** Code quality and style will be automatically verified using GitHub Actions CI/CD pipelines. This level is met if code contributions pass automated static analysis checks.

Project

Contents

P.1 Roles and personnel 41
P.2 Imposed technical choices 43
P.3 Schedule and milestones 43
P.4 Tasks and deliverables 44
P.5 Required technology elements 46
P.6 Risks and mitigation analysis 47
P.7 Requirements process and report 47

P.1 Roles and personnel

Project responsibilities are structured across the team, ensuring that every member participates in meeting, technical, and research roles while maintaining a primary focus on their assigned responsibilities.

Andreas

- **Meeting Role (Chair):** Generally runs meetings, taking lead on agenda creation and keeping the team focused to cover all action items during meeting.
- **Technical Role (CI/CD Engineer):** Focuses on creating workflows to support CI and CD.
- **Research Role (Qualitative Research Lead):** Conducts/designs post-test interviews, surveys and focus groups.
- **Personal Background and Role Fit:** Is a full time McMaster student in Software Engineering able to dedicate 10+ hours/week to the project. Has programming experience in Java and Python, and experience with engineering documentation in industry

Andy

- **Meeting Role (Scribe):** Records everything discussed, updating tasks, deadlines, and next steps/action items on the GitHub Repository.

- **Technical Role (Systems Architect):** Takes lead on architecture design and oversees development is according to the blueprint.
- **Research Role (Ethics and Participant Coordinator):** Drafts consent forms and helps recruit study participants.
- **Personal Background and Role Fit:** Is a full time McMaster student in Software Engineering able to dedicate 10+ hours/week to the project. Prior experience with developement (3+ years, 2+ game jams), has worked extensively with GitHub Version Control, Developer UI via Web development, and has documented game descriptions as prior job.

Chris

- **Meeting Role (Progress Tracker):** Follows up on details from previous meetings and deadlines to ensure team stays on track with older tasks.
- **Technical Role (Back-end Lead):** Leads development of the back-end of the program.
- **Research Role (Research Design Lead):** Formulates the research questions, hypotheses, and experimental methodology for evaluating the user experience.
- **Personal Background and Role Fit:** Is a full time McMaster student in Software Engineering able to dedicate 10+ hours/week to the project. Minor experience in game development through own projects in Unity.

Humna

- **Meeting Role (Quality/Scope Assessor):** Assesses the scope of each decision made and ensures that team stays within the desired workload.
- **Technical Role (Front-end Lead):** Leads development of the front-end of the program.
- **Research Role (Statistical Data Analyst):** Analyzes quantitative data to drive further decisions.
- **Personal Background and Role Fit:** Is a full time McMaster student in Software Engineering able to dedicate 10+ hours/week to the project. Has previous software development experience through internships and academic projects, including working with Java, Python, JavaScript, and user interfaces. This experience will be useful for front-end development and analyzing data during usability testing.

Owen

- **Meeting Role (Risk Assessor):** Should question all assumptions made and attempt to poke holes in all decisions, should try and get team to think deeply about everything.

- **Technical Role (Front-end Lead):** Leads development of the front-end of the program.
- **Research Role (Data Collection Analyst):** Builds logging mechanisms into the software to capture quantitative user metrics (time-played, score, etc.).
- **Personal Background and Role Fit:** Is a full time McMaster student in Software Engineering able to dedicate 10+ hours/week to the project. Executive member of the McMaster Game Development Club, has experience with game design/development (Organized and/or participated various game jams) and UI/UX.

Dr. Welch

- **Supervisor/Advisor:** Provides high level and technical guidance, overseeing general project direction, and aiding with methodologies.
- **Personal Background and Role Fit:** Assistant Professor in the Department of Computing and Software with expertise in natural language processing, personalization and specialization in computing methodologies.

P.2 Imposed technical choices

- **Game Engine**: The project will entail the use of a game engine to aid in development, since it is common practice in game development and was decided upon during the development plan. A game engine provides lots of supports such as an libraries, visual development tools, and other useful features tailored to game development. This will act as the main interface for developing the game.
- **Coding Language**: Any given game engine only supports one or two languages for use within the engine. For example, using the game engine Godot will constrain the project to either GDScript or C#, with GDScript having more support and documentation from the engine.
- **Github Actions**: The project must implement Github Actions to create a CI/CD pipeline during development since the Capstone course requires automation through Github Actions.
- **Hardware Options**: The project is required to perform usability testing, and it was decided that as a point of variation more than one input method must be accomodated. This will be provided in order to accomodate more users' preferences.

P.3 Schedule and milestones

The project's primary schedule will follow the milestones and dates below, each named according to the deliverables they must produce (as discussed in the project's development plan):

- M1 Problem Statement, Development Plan, PoC Plan, and Team Charter (Due by September 28, 2026)
- M2 SRS and Hazard Analysis Revision 0 (Due by October 19, 2026)
- M3 VnV Plan Revision 0 (Due by November 2, 2026)
- M4 Design Document Revision -1 and CD-Fall (FODA) (Due by November 16, 2026)
- M5 PoC Demo (Due by November 23, 2026)
- M6 Design Documentation Revision 0 (Due by January 18, 2027)
- M7 Revision 0 Demo (Due by February 1, 2027)
- M8 VnV Report Revision 0 and CD-Winter Revision 0 (Usability Report) (Due by March 8, 2027)
- M9 Final Demonstration Revision 1, EXPO Demonstration (Due by March 15, 2027)
- M10 Final Documentation (Revision 1) (Due by March 31, 2027)

P.4 Tasks and deliverables

The tasks required to be completed to develop RealmForge (and its feature variation capabilities) will be managed using an iterative and incremental approach. Within each milestone (in this context the milestones associated with source code development), team members will divide the work to be done into suitable issues, viewable in RealmForge's GitHub Project. The issues will be coordinated using a Kanban method, tracking their progress and review status. The software features associated with these issues will be regularly planned, developed, reviewed, tested, and deployed using a feature branch workflow, contributing towards each set milestone.

A schedule of development deliverables pertaining to the milestones listed above is available below, complete with relevant requirement sets that are planned to be implemented at the completion of each deliverable. Milestones that purely involve documentation development are omitted, as this schedule is intended to outline the completion of software development tasks and their associated requirements.

Detailed Schedule:

- M5 PoC Demo
  - **Deadline:** November 23, 2026
  - **Software Deliverables:** RealmForge Proof of Concept Build

        * **Associated Software Tasks:**
        * Review PoC plan and requirements
        * Develop PoC features
        * Review implemented PoC features on a recurring basis during development
        * Test PoC features
        * Verify associated requirements (outlined below)
        * Deploy PoC
    - **High Priority Requirements:**
        * SFR-006
        * SFR-011 - 013
        * SFR-015 - 016
    - **Requirements Implemented:**
        * SFR-003
        * SFR-006
        * SFR-009 - 013
        * SFR-015 - 016

- M7 Revision 0 Demo
    - **Deadline:** February 1, 2027
    - **Software Deliverables:** RealmForge Revision 0 Build
        * **Associated Software Tasks:**
        * Review Revision 0 design documentation and requirements
        * Develop Revision 0 features
        * Review implemented Revision 0 features on a recurring basis during development
        * Test Revision 0 features
        * Verify associated requirements (outlined below)
        * Deploy Revision 0
    - **High Priority Requirements:**
        * SFR-004
        * SFR-007 - 008
        * SFR-014
        * SFR-017

    * SFR-018 - 019
    * SFR-021 - 022
    * SFR-023 - 027
    * SNFR-002 - 005
    * SNFR-007
    * SNFR-009
- **Requirements Implemented:**
    * Requirements implemented by the PoC
    * SFR-001 - 002
    * SFR-004 - 005
    * SFR-007 - 008
    * SFR-014
    * SFR-017 - 028
    * SNFR-001 - 009

P.5 Required technology elements

The RealmForge software will be run, developed, and tested using hardware that supports Windows 10/11 OS. Other OS's such as Apple's or other platforms will be considered after the initial development showcase. Inputs are handled either with a standard keyboard and mouse or with a connected controller using standard button layout (analog sticks, d-pads, shoulder and 4-buttons layout). The controller should be compatible with any Windows OS, similarly to the mouse and keyboard. The game output will be displayed using any standard monitor connected to the computer hardware.

The system will be built using the Godot game engine, including using Godot's own coding language, GDScript. Godot is an open-source application and has an easy learning curve to implement features, which is a main concern regarding this project. The easier the team can fully pick up the engine, the faster and cleaner it is to create a working concept. Each team member will choose their own IDE. GDUnit4 and GDLint and other external third-party modules will be used for CI/CD as a safeguard for good coding practices and coverage. The project will use GitHub's version control and cloud storage for the source code and documents. The GDLint and GdUnit4 unit tests and linting will be implemented using GitHub Actions available on the GitHub website; this is so that errors and bugs will be caught and reiterated early on. LaTeX will be used for writing important documents such as Hazard Analysis, Development, etc. Issues about the project will be raised on the GitHub website and written/handled with its own built-in issue tracker. Other important project management tasks will be written using GitHub Project's Kanban board for organization and easy tracking.

P.6 Risks and mitigation analysis

- Risk: High Amount of Features in Scope
  - Description: Scope creep could increase development time to the point where deadlines are not met/up to standard. This can create rushed features which cause an unpleasant user experience through poor game design.
  - Mitigation: The team will use the GitHub Kanban board to track features and thoroughly inspect each new feature to be added. This will allow discernment of how much development time will be added from all aspects. The team will also ensure each feature added is up to the team's standard before building more features on top of them, lowering scope if necessary.
- Risk: Issues with Godot
  - Description: Technical issues may arise with Godot, the selected game engine. The team is only somewhat experienced with Godot, making these issues difficult to troubleshoot.
  - Mitigation: The team will collaboratively address issues when they arise, and will make use of Godot's thorough documentation to aid in troubleshooting. The team will keep Godot inexperience in mind, and if a feature proves too complicated to implement, alternatives will be sought, such as audio/visual assets able to be used or Godot third-party modules. If this is not possible, scope will be lowered accordingly.
- Risk: Usability Testing Data Quality
  - Description: Usability testing will be conducted in order to discern player preferences, and may not yield data of high enough quality/quantity if turnout for elicitation is lower than expected.
  - Mitigation: The team plans to conduct usability testing on various relevant McMaster clubs that team members are a part of, such as the McMaster Game Development Club or McMaster Geeks. This will demonstrate higher volunteer turnout for the testing than conducting it without this assistance.

P.7 Requirements process and report

Comment: *Initially, description of what the requirements process will be; later, report on its steps.*

Appendix

Appendix — Generative AI Use Disclosure

AI Tools Used

- OpenAI GPT-6
- Google Search AI Overview (Google Gemini)

How and Where Used

AI tools were used for assistance with LaTeX syntax and commands, during the development of sections E2, E3, S4, P3, and P4, and when developing LaTeX formatting for the document as a whole. Specifically, AI tools were used for assistance in creating the LaTeX custom requirement format macro used throughout the document, as well as resolving various minor formatting and table of contents bugs. Additionally, these tools were also used in these sections to conduct brief 'sanity checks' of content in accordance with Meyer's requirements document template (the template description was provided). These 'sanity checks' did not substitute full reviews of the sections, and were more used as a brief second opinion. In these sections, AI tools were not used to generate any text or wholly generate any of the ideas or conclusions presented.

Verification of AI-Generated Content

Any LaTeX syntax or code provided by AI tools was verified by testing compilation in LaTeX documents, and ensuring the resulting output was correctly formatted and matched the needs of the document. Conclusions from brief 'sanity checks' of the above sections using AI tools were independently verified for correctness using course materials and documentation of Meyer's requirements template, and no ideas or text were newly generated by AI tools and presented in the document.

Appendix — Reflection

Reflection Questions Answered:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?
4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.
5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.
6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Andreas:

1. When writing this deliverable, adopting Bertrand Meyer's requirement template and completing its sections was helpful and applicable to our project, since all team members were familiar with the template previously, and it is provided with thorough documentation. Understanding the template previously, and being able to review its full documentation helped our team to understand what was required to complete each section. Additionally, writing this deliverable forced our team to deeply consider various details of the project and requirements that the system must satisfy, which helped to enable us to make important project decisions. Obviously one must decide on various project details when writing a requirement specification, but completing this SRS helped us to solidify and exactly outline the outputs needed to be produced by the system, and other constraints it must follow.
2. One pain point I experienced during this deliverable was coordinating the many sections to be written and decisions to be made as a group. Ideally, all project and system planning decisions will be made together, but work must be divided at some point so that each team member can asynchronously complete their tasks (while reviewing and discussing with the team when needed). This at times led to more

administrative overhead work, as various sections needed to be iteratively reviewed and discussed to ensure that the document did not contain major contradictions. However, this coordination work was required for the success of the document, and was lessened in certain scenarios by initially discussing clear expectations for various sections as a group, to ensure that the team member completing a specific section had the information required to do so correctly.

3. A majority of the sections I contributed to did not involve the production of formal requirements, except E.3 (Constraints), which included formal environmental constraint requirements. Some of these requirements (feature variation points, and user input device requirements) were primarily influenced by discussions with our TA and professor, using their recommendations on including feature/system variability in the project, and specifically doing so by having players use different input methods (controller, keyboard and mouse). Additionally, the environment requirement included concerning the operating system RealmForge will support was influenced by discussions with peers that could be included in our user groups (those who play video games and are familiar the roguelite genre, those who are not, etc.). It was found (as expected) that most users tend to play PC video games on machines running modern Windows operating systems, as they are most commonly available and easy to configure to play popular video games.

4. Of the courses I have taken (and are currently taking), SFWRENG 3RA3 (Requirements) and 4HC3 (Human-Computer Interaction) are particularly helpful to our team (and myself specifically) for our capstone project. SFWRENG 3RA3 introduced all team members to the use of Meyer's requirements template and principles of requirements engineering as a whole, which helped us to complete this document, and construct requirements for the system. SFWRENG 4HC3 provides our team with important knowledge and techniques for effective user interface and experience design, which will be particularly helpful for our project, since it primarily focuses on interface design and usability testing.

5. One skill the team will collectively need to acquire (or further develop) is video game development. This will include becoming more familiar with our chosen game development engine and language, as well as better understanding game development principles. This will be especially important for myself, as I have limited experience in programming for video game development. Another area of knowledge that will need to be further developed for each team member concerns usability testing techniques and principles, as each of us are just beginning to study these concepts in SFWRENG 4HC3 (as discussed above).

6. For improving game development skills, team members could use provided game engine tutorials and documentation for the game engine used, review YouTube and web article lessons on game development principles, or read books that provide instruction on game development principles. I will individually aim to primarily follow tutorials and documentation for our chosen game engine, since the initial

technical skills of game development are new to me. I will also aim to improve my understanding of game development principles through various tutorials and lessons when possible. For developing interface design and usability testing knowledge, team members could take notes and review how to best apply the concepts taught in SFWRENG 4HC3 to our Capstone project specifically, or consult other standards or tutorial sources for interface design and usability testing. I will primarily aim to review and understand how the concepts taught in 4HC3 could be applied to our project, as I currently am developing my understanding of these concepts well in the context of the course, and believe they could be effective when applied to the development of RealmForge.

Owen:

1. When writing this deliverable, collaborating using the Meyer template made the process much easier since everyone had worked with it previously in the course SFWRENG 3RA3. The division of work went smoothly at the beginning as well, and communication throughout the process made us get more specific with what exactly implementation would entail, what constraints/requirements exist, and how those will be satisfied.

2. There were some pain points throughout the deliverable. The first one was the adjustment to using branches much more often (as opposed to a branch for each document, it was per section). This took a little bit of getting used to for me to do this without issue, but was resolved by discussing the process with group members, and getting comfortable with the process of branch management. Another pain point was consistent communication, as the bulk of the work was done during reading week, when people's schedules are at their most inconsistent. This was also made worse by many sections being written individually, without realizing at the time of writing that certain sections are similar, which created some redundant points. This was resolved by constantly discussing clear guidelines for sections to ensure everything is cohesive and not repetitive. This preventative measure started once the issue was noticed, and ensured more members knew exactly what was required of them.

3. Basically all requirements I had a hand in writing were directly inspired by speaking to clients/stakeholders, although not always in the context of the project. I have had plenty of discussions about game design with friends and peers that apply to the project and its requirement elicitation, since a lot of friends of mine like games and are interested in game development like I am. Although many of these people are potential users/stakeholders of the project, most of these conversations either predate the project's inception or were not directly related. The exception is our TA and supervisor, both of whom are experienced with video games and stakeholders in the project.

4. I can think of many courses, past and present, that will help the team's success with the capstone project.

- SFWRENG 4HC3: This course is teaching about how to build effective user interfaces and experiences, which will help build usable knowledge for the project since UI and various design principles described in this course are all very important to the success of this project.
- SFWRENG 3RA3: As mentioned in question 1, the group all has experience with the Meyer SRS template introduced in this course and also gained important information about requirements from this course.
- SFWRENG 3S03: Software testing knowledge from this course will be integral to how we approach the project.
- SFWRENG 2XC3: This course was an introduction to many core concepts we use in this project, such as version control, testing, debugging, and CI/CD.

5. The main skill that the team as a whole will need to acquire and develop is video game development and its intricacies. Since I am somewhat knowledgeable about this, I will do what I can to teach the group as needed about the game engine and language we use. Another skill the team will need is setting up test suites and linting. This is at least a requirement for myself, as I am not very experienced with setting these things up, especially in conjunction with a game engine. The practice of good UI design is something being taught in 4HC3 as mentioned in Q4, and is important for the team to absorb from the course.

6. Game Development skill acquisition approaches:
   - Following online tutorials to understand the engine/language and game development techniques, either on YouTube or provided by the engine's documentation.
   - Reading up on documentation to learn what is possible in the chosen engine and language.
   - Discuss with peers or members of the group with more experience with game design and the chosen engine/language.

   Testing and Linting skill acquisition approaches:
   - Experimenting with these tools in a neutral/sample environment.
   - Following online tutorials to set up/use the chosen tools in the chosen game engine.

   Good UI Design skill acquisition approaches:
   - Following along with and drawing from SFWRENG 4HC3 as we are taught more (or consulting a peer in that class, if not taking the class yet) and participating in assignments to build skills.
   - Experimenting with UI setup through YouTube tutorials, 4HC3 knowledge, or other means, tweaking the design based on what you personally like (and testing it on peers).

Humna:

1.

2.

3.

4.

5.

6.

Christopher:

1.

2.

3.

4.

5.

6.

Andy:

1. The team was very active during the process overall. Almost every section had a long discussion; if there was an issue concerning the content of the section then the group members explicitly tried to clarify with one another to clarify the misunderstanding or issue. Each document section was iterated at least twice for qualities sake; one for a rough draft and another for a good draft. The group maintained the organization of the documents and there was little or no conflicts during each GitHub push. Finally, each member responded quickly whenever they needed to revise their parts. The team was proactive and left no part of the document incomplete or lacking in depth.

2. I felt there was pain point regarding the prioritizing of certain sections. Some sections just needed to finish or start before some others because some parts relied on or needed some framework before starting. Had the group given some milestones on which article was completed first then we would not have some major delays.

3. Almost all the requirements were discussed amongst my peers. For external requirements like usability and reports, the requirement was discusses with the teaching assistant, professor and the potential users.

4. The courses 3RA3 software requirements and 4HC3 visual interfaces will help. The 3RA3 directly helps with writing all documents including this one. The 4HC3 course helps with getting an idea on how to obtain vital feedback from stakeholders and users. The visual interface also gives me a clear base knowledge of the standards of user interface and visual appeal.

5. The team must learn to at least make a very easy game via the Godot tutorials. Major GitHub base knowledge should be improved as the team keeps using the technology. Intricate knowledge of the CI/CD pipeline via GitHub Action will be needed for major coding standards to be implemented and discussed.

6. The basic Godot knowledge should be learned with either online video tutorials or using the official Godot documentation website. The GitHub Actions and base knowledge should be improved on supervisor knowledge or again using the official online Git website. I would proceed with learning the CI/CD GitHub Actions pipeline while the rest of the team should pursue the Godot and base GitHub knowledge. We discuss this during the role assignment phase of the previous milestone.